

Vector Embeddings for File-Based Document CRUD System

A design doc for adding vector search to a multi-format (md/toml/yml/txt) CRUD system that needs to work across teams via git, run on practically any hardware, and stay simple to ship.

The Core Constraint: Git-Mergeability

The single most important architectural decision is driven by one fact:

SQLite databases do not merge in git.

Two contributors editing different documents will produce conflicting `.db` files that git cannot reconcile. Whoever pushes second has to regenerate. With Git LFS this becomes a hard conflict where one side's work is lost unless manually rebuilt.

This rules out SQLite as the *shared* embedding store for a repo-based workflow, even though it's an excellent choice for solo/local use.

Recommendation: Sidecar Files

Store embeddings as small files that shadow the source documents:

```
docs/  
  api.md  
  auth.md  
.embeddings/  
  docs/  
    api.md.vec  
    auth.md.vec
```

When Alice adds embeddings for `api.md` and Bob adds embeddings for `auth.md`, git merges the result trivially because they are separate files. This matches how the system already thinks about documents.

Sidecar File Format

Recommended: **gob** (Go-native) or **msgpack** (cross-language). Both are compact, fast to

deserialize, and binary-safe.

Each sidecar contains:

Field	Purpose
source_hash	SHA-256 of the source file content. Detects staleness on pull.
model_id	e.g. nomic-embed-text:v1.5-q4_0 . Detects model mismatch.
model_dim	Vector dimensionality. Sanity check.
created_at	Timestamp for debugging.
chunks	Array of {text, start_offset, end_offset, vector} .

Format trade-offs:

- **gob / msgpack** — compact, fast, binary. Best balance for this use case.
- **JSON** — diffable-ish, but 3–4x larger and slower to parse. Fine for tiny corpora.
- **Parquet** — overkill for one-doc-per-file; shines when batching.

Hot-Load Path (Ephemeral Docs)

For unstructured or session-only documents, use the same embedding code path but skip the sidecar write. Hold vectors in memory for the session.

A 5–10 MB markdown file chunked at ~500 tokens yields roughly 50–200 chunks. With a small model running on Ollama, embedding takes well under a second on modern hardware, perhaps 1–2 seconds on older laptops. The embedding call is the bottleneck, not the storage.

Search Strategy

Load all sidecars into memory at startup. Brute-force cosine similarity is genuinely fast at the scales most projects will hit:

Corpus size	Memory (384-dim)	Search time
1k chunks	~1.5 MB	< 1 ms
10k chunks	~15 MB	2–5 ms

100k chunks	~150 MB	30–80 ms
-------------	---------	----------

Only reach for an HNSW index (e.g. `coder/hnsw`) when the corpus exceeds ~100k chunks or memory becomes a concern.

Model Selection: Ollama with a Light Model

Ollama is the right call for a multi-platform team — single binary, runs anywhere, easy model management. Recommended models for the “works on practically any hardware” constraint:

Model	Size	Dim	Notes
<code>all-minilm</code>	~45 MB	384	Fastest, lower quality. Good for keyword-adjacent search.
<code>snowflake-arctic-embed:xs</code>	~90 MB	384	Strong quality-for-size.
<code>nomic-embed-text</code>	~270 MB	768	Best quality. Worth it if contributors can download once.

Determinism Caveat

Embedding outputs can vary *slightly* across hardware due to floating-point non-determinism — different CPU SIMD paths, GPU vs CPU, quantization differences between Ollama versions. For **search quality** this is irrelevant; cosine similarity is robust to tiny perturbations. For **bit-exact reproducibility** of stored vectors, it is not guaranteed.

Mitigations:

1. Pin exact model + quantization in config (e.g. `nomic-embed-text:v1.5-q4_0`).
2. Store the model identifier in each sidecar.
3. On model mismatch, regenerate rather than trying to reconcile.
4. Accept that vectors may shift slightly between contributors and let git store the new bytes — semantic search results will be effectively identical.

Staleness Detection

On every load (or on a `verify` command), compare the source file’s current hash to the `source_hash` stored in its sidecar:

- **Match** → use cached vectors.
- **Mismatch** → source was edited without regenerating; warn the user and offer to recompute.
- **Missing sidecar** → new doc; generate on first use or via batch command.

This same mechanism handles the “didn’t rebase before updating” scenario gracefully — after a pull, any docs with stale sidecars are flagged.

Git Workflow Behavior

Scenario	SQLite	Sidecars
Two contributors edit different docs	Binary conflict	Clean merge
Two contributors edit the same doc	Binary conflict, manual rebuild	File conflict, take either (semantically equivalent) or regenerate
Contributor forgets to pull before pushing	DB diverges, painful recovery	Standard git conflict per file
Reviewing a PR	Opaque blob	See which docs gained/changed embeddings
Repo size growth	Monolithic, grows with every change	Granular, only changed sidecars touched

Sidecars win on every axis that matters for collaboration.

Optional: Dual-Mode Persistence

The embedding generation logic is identical regardless of where vectors are stored. Consider exposing a config option:

- **backend: sidecar** — file-per-doc, mergeable, default for repo-shared use.
- **backend: sqlite** — single-file DB with `sqlite-vec` (via `ncruces` for cross-platform), faster queries, recommended for solo/local use.

The abstraction cost is small and gives users the right tool for their context.

Suggested Stack

1. **Embedding generation:** Ollama + `nomic-embed-text` (or `all-minilm` for the lowest-end hardware).
2. **Persistence (default):** gob or msgpack sidecar files mirroring the doc tree under `.embeddings/`.
3. **Persistence (optional):** SQLite via `ncruces/go-sqlite3` with `sqlite-vec` for solo/local mode.
4. **Search:** in-memory brute-force cosine similarity until the corpus exceeds ~100k chunks.
5. **Integrity:** SHA-256 of source content + model ID stored in every sidecar; verify on load.
6. **Hot-load:** same pipeline, skip the persistence write, vectors live in process memory for the session.

Open Questions Worth Deciding Early

- **Chunking strategy.** Fixed-size token windows? Heading-aware for markdown? Schema-aware for toml/yml? This affects sidecar size and search quality more than the storage format does.
- **Should `.embeddings/` be gitignored by default?** Committing it gives newcomers instant search; ignoring it keeps the repo lean but requires every contributor to regenerate. A `embeddings: tracked | local` config flag could expose this.
- **Batch regeneration UX.** A `regen --stale` command that walks the tree, hashes sources, and rebuilds anything out of sync is probably the most-used workflow.